

SMART LABORATORY EQUIPMENT FAULT MANAGEMENT

*A project report submitted in
partial fulfillment of
the requirements for the award of the degree of*

BACHELOR OF TECHNOLOGY

in

computer science & engineering

from

APJ ABDUL KALAM KERALA TECHNOLOGICAL UNIVERSITY



Submitted By

SANA ASHARAF (KITM23CS038)



KMCT Institute Of Technology And Management

Department of Computer Science and Engineering

Kuttippuram, Malappuram, Kerala-679571

MAR 2026

Department of Computer Science and Engineering

KMCT Institute Of Technology And Management

Kuttippuram, Malappuram, Kerala – 679571



CERTIFICATE

This is to certify that the Mini Project report entitled "Smart Laboratory Equipment Fault Management System" is a bonafide record of the work done by SANA ASHARAF (KITM23CS038) under our supervision and guidance. The report has been submitted in partial fulfillment of the requirement for award of the Degree of Bachelor of Technology in Computer Science and Engineering from the APJ Abdul Kalam Kerala Technological University for the year 2026.

Ms. Varsha K
Project Guide
Assistant Professor
Dept. of Computer Science and Engineering
KMCT Institute of Technology and
Management

Ms. Harshyanas M
Head of department
Dept. of Computer Science and Engineering
KMCT Institute of Technology and
Management

ACKNOWLEDGEMENT

We express our sincere gratitude to all those who have supported us throughout the course of this mini project. First and foremost, we thank the Almighty for blessing us with the wisdom, strength, and perseverance to complete this project successfully.

We would like to express our heartfelt thanks to our project guide, Ms. Varsha K, Assistant Professor, Department of Computer Science and Engineering, KMCT Institute of Technology and Management, Kuttippuram, for her invaluable guidance, constant encouragement, and constructive suggestions throughout the development of this project. Her expertise and insightful feedback were instrumental in shaping the direction of our work.

We are deeply grateful to the Head of the Department and all the faculty members of the Department of Computer Science and Engineering for their continuous support, motivation, and technical guidance. Their encouragement has been a great source of inspiration throughout our academic journey.

We would also like to thank the Principal of KMCT Institute of Technology and Management for providing us with the necessary facilities and a conducive environment to carry out this project.

Our sincere thanks are extended to the lab staff and technical personnel of the institution, whose experiences with manual fault reporting processes inspired the conceptual foundation of this project. Their inputs regarding the real-world challenges of laboratory equipment maintenance were invaluable in defining the problem statement.

We are also thankful to our parents and family members for their unwavering support, patience, and encouragement throughout this endeavour. Their moral support has been a constant source of strength.

Finally, we would like to thank all our friends and classmates who provided assistance, feedback, and motivation at every step of this project. This work would not have been possible without the collective support of everyone mentioned here.

ABSTRACT

The Real-Time Laboratory Equipment Fault Management System is designed to improve the process of reporting and managing lab equipment faults in educational and research institutions. Traditional fault reporting methods, such as phone calls and manual registers, are slow and inefficient, leading to delays in issue resolution and disruption of laboratory activities. This paper reviews related works in the domain of fault tracking, computer maintenance management, and complaint handling systems, and proposes a centralized digital platform that allows users to report faults, track resolution status in real time, and receive automated updates.

The proposed system includes features such as automated task assignment, role-based access control, real-time notifications, cluster fault reporting, predictive maintenance suggestions, QR-code based equipment identification, and an administrative dashboard for oversight. A comparison of six related works reveals that no existing system integrates all of these capabilities in a single framework tailored for laboratory environments. The proposed system addresses this gap by combining a web application built with Next.js and a mobile application built with React Native, Node.js, and PostgreSQL, delivering a comprehensive, scalable solution for laboratory fault management.

Keywords: Fault Management System, Laboratory Equipment, Real-Time Tracking, Web Application, Mobile Application, Role-Based Access, Automated Notifications, Maintenance Management, QR Code, Predictive Maintenance

TABLE OF CONTENTS

CHAPTER / SECTION	PAGE NO.
Acknowledgement	ii
Abstract	iii
Content	iv
List of Abbreviations	v
List of Figures	vi
List of Tables	vii
CHAPTER 1: INTRODUCTION	
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope of the Project	3
1.5 Organization of the Report	5
CHAPTER 2: LITERATURE SURVEY	
2.1 Review of Related Works	6
2.2 Comparative Analysis	10
2.3 Research Gap	10
CHAPTER 3: SYSTEM DESIGN	
3.1 System Overview	12
3.2 User Roles and Access Control	12
3.3 Use Case Workflow	13
3.4 System Architecture	15
3.5 Database Design	17
CHAPTER 4: METHODOLOGY AND IMPLEMENTATION	
4.1 Important Parameters	24
4.2 Architecture	26
4.3 Implementation	28
4.4 Results and Discussion	30
CHAPTER 5: PROPOSED SYSTEM FEATURES	
5.1 Key Features	42
5.2 Technology Stack	44
5.3 Advantages and Limitations	44

5.4 Recommended Development Order	45
-----------------------------------	----

CHAPTER / SECTION	PAGE NO.
CHAPTER 6: CONCLUSION AND FUTURE WORK	
6.1 Conclusion	46
6.2 Future Work	47
References	49

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
QR	Quick Response
CMMS	Computerized Maintenance Management System
CMMIS	Computerized Maintenance Management Information System
UI	User Interface
API	Application Programming Interface
REST	Representational State Transfer
DB	Database
RBAC	Role-Based Access Control
JWT	JSON Web Token
SQL	Structured Query Language
PK	Primary Key
FK	Foreign Key
CSS	Cascading Style Sheets
JS	JavaScript
TS	TypeScript
URL	Uniform Resource Locator
HTTPS	Hypertext Transfer Protocol Secure
CSE	Computer Science and Engineering
KTU	APJ Abdul Kalam Technological University
IoT	Internet of Things
AI	Artificial Intelligence
ML	Machine Learning
PDF	Portable Document Format
ENT	Equipment

LIST OF FIGURES

FIGURE NO.	FIGURE TITLE	PAGE NO.
Figure 3.1	Complete System Workflow Diagram	20
Figure 3.2	User Flow - QR Based Lab Issue Tracking	20
Figure 3.3	Admin Flow - Dashboard and Issue Management	21
Figure 3.4	Technician Flow	21
Figure 3.5	QR Code Scanning Internal Flow	22
Figure 3.6	System Architecture Overview	22
Figure 3.7	Client-Server Architecture Diagram	23
Figure 3.8	ER Diagram - Database Relationships	23
Figure 4.1	Use Case Diagram	33
Figure 4.2	QR-Based Complaint Registration Flow	33
Figure 4.3	Complaint Lifecycle Status Flow	34

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE NO.
Table 1.1	Comparison of Related Works vs Proposed System	10
Table 1.2	Literature Survey Summary	11
Table 3.1	Users Table - Database Schema	17
Table 3.2	Labs Table - Database Schema	17
Table 3.3	Systems Table - Database Schema	18
Table 3.4	Issue Reports Table - Database Schema	18
Table 3.5	Technician Assignments Table - Database Schema	19
Table 3.6	Maintenance History Table - Database Schema	19
Table 3.7	Notifications Table - Database Schema	19
Table 3.8	Activity Logs Table - Database Schema	19
Table 4.1	Advantages of the Proposed System	32
Table 4.2	Limitations of the System	32
Table 5.1	Technology Stack - Frontend and Backend	44
Table 5.2	Project Modules and Purpose	45

CHAPTER 1

INTRODUCTION

1.1 Background

In any academic or research institution, laboratory equipment forms the backbone of practical learning and experimentation. As the number of instruments and devices in use grows, so does the frequency of faults, breakdowns, and maintenance requirements. Managing these faults efficiently is critical to ensuring that teaching activities continue without interruption and that costly equipment remains in good working order.

Today, most institutions rely on manual or informal processes for fault reporting. A student or faculty member who encounters a broken projector, a malfunctioning oscilloscope, or a damaged computer must either call the laboratory administrator, fill out a paper form, or simply leave a note. These approaches are unreliable. Reports get lost, priorities are not set systematically, technicians are dispatched without full information, and administrators have no visibility into recurring problems or overall equipment health.

The evolution of digital infrastructure management has opened new avenues for addressing these challenges. Web-based platforms, mobile applications, QR-code identification systems, and cloud-hosted databases now provide the technical building blocks needed to design a comprehensive, real-time fault management solution tailored specifically to laboratory environments.

The Smart Laboratory Equipment Fault Management System proposed in this report replaces the fragmented, informal process with a unified digital solution. Users can report faults from their phones or computers, select the affected equipment through a QR-code scanner, describe the problem, and upload supporting photographs. The system automatically assigns the fault to the most appropriate technician, tracks every stage of resolution, and notifies all relevant parties when the status changes. Administrators can view a live dashboard showing open faults, technician assignments, and historical data, and can generate reports for institutional records.

1.2 Problem Statement

The current approach to laboratory fault management in educational institutions suffers from four primary weaknesses that collectively undermine the efficiency and effectiveness of maintenance operations:

1. Informal and Inconsistent Fault Reporting:

Fault reporting relies on verbal communication, phone calls, paper forms, or WhatsApp messages, with no standard channel or format. This makes it difficult to ensure that problems are actually recorded, properly categorized, and acted upon. Critical information about the fault, the affected equipment, and the urgency of the situation is frequently lost or misrepresented during this informal transfer.

2. Absence of Real-Time Visibility:

Once a fault is reported through informal channels, the reporting user has no mechanism to track whether the report was received, whether a technician has been assigned, or what stage of resolution the issue has reached. This opacity creates frustration among users and prevents administrators from maintaining an accurate picture of current maintenance demands.

3. Inefficient Technician Assignment:

Task assignment to technicians is performed manually by administrators without systematic consideration of technician workload, availability, or area of specialization. This leads to uneven workload distribution, duplicated effort when multiple users report the same fault, and delays in addressing high-priority issues while lower-priority tasks consume technician time.

4. Lack of Data-Driven Maintenance Planning:

No mechanism exists to systematically analyze fault data over time. Administrators cannot identify equipment that fails repeatedly, cannot track resolution times or technician performance, and cannot use fault history to make informed decisions about preventive maintenance schedules, spare parts procurement, or equipment replacement priorities.

1.3 Objectives

The primary objectives of the Smart Laboratory Equipment Fault Management System are as follows:

- To design and implement a centralized, cloud-based platform for reporting, tracking, and resolving laboratory equipment faults in educational institutions.
- To enable instant and accurate equipment identification through QR-code based scanning, eliminating manual data entry errors during fault reporting.
- To implement role-based access control that provides tailored interfaces and appropriate permissions for students, faculty, technicians, and administrators.
- To automate the fault prioritization and technician assignment process based on equipment type, fault category, technician availability, and current workload.
- To provide real-time notifications to all stakeholders at every stage of the fault resolution lifecycle, from submission through assignment to resolution.
- To develop a comprehensive administrative dashboard that provides centralized visibility into complaint statistics, system health indicators, laboratory-wise issue distribution, and technician performance metrics.
- To implement an analytics module that generates actionable insights from historical fault data to support preventive maintenance planning and informed resource allocation.
- To design a scalable and maintainable system architecture using modern web technologies including React.js, TypeScript, Supabase, and PostgreSQL.
- To deploy the system on cloud infrastructure to ensure high availability, remote accessibility, and operational continuity without the need for dedicated institutional servers.
- To address the research gap identified in the literature by providing the first integrated system that combines all critical fault management features in a framework specifically designed for laboratory environments.

1.4 Scope of the Project

The scope of the Smart Laboratory Equipment Fault Management System encompasses the following operational domains:

Institutional Coverage:

The system is designed to serve educational institutions with multiple laboratories, office environments, and technical staff members. It supports any number of laboratories, computer systems, complaints, and administrative users within the institution.

User Management:

The system manages four distinct user categories: Students (fault reporters), Faculty (fault reporters), Technicians (maintenance personnel), and Administrators (oversight and management). Each category has defined access permissions and a tailored operational interface.

Equipment Management:

The system manages information about individual computer systems and laboratory equipment, including their location, operational status, maintenance history, and QR-code identifiers. It supports an unlimited number of registered equipment items.

Fault Lifecycle Management:

The system manages the complete lifecycle of each fault report from initial submission through technician assignment, active maintenance, and final resolution. Every stage is timestamped, tracked, and made visible to all relevant stakeholders.

Analytics and Reporting:

The system provides real-time analytics on complaint distribution, resolution rates, technician performance, and laboratory health indicators. Administrative users can generate reports filtered by date range, equipment category, technician, or laboratory.

1.5 Organization of the Report

This report is organized into six chapters. Chapter 1 provides the introduction, background, problem statement, and objectives. Chapter 2 presents a detailed literature survey of six related works and a comparative analysis that establishes the research gap. Chapter 3 covers

the system design including user roles, workflow diagrams, architecture, and database schema. Chapter 4 describes the methodology and implementation including important parameters, technical architecture, and results. Chapter 5 presents the proposed system features, technology stack, and advantages. Chapter 6 concludes the report with a summary of findings and directions for future work.

CHAPTER 2

LITERATURE SURVEY

A systematic review of six relevant works was conducted to understand the current state of the art in fault reporting, computer maintenance management, and complaint tracking systems. The objective was to identify the strengths and limitations of existing approaches and to determine where existing solutions fall short of addressing the complete requirements of a laboratory fault management system.

2.1 Review of Related Works

2.1.1 Zetian Guo and Yingyue Chen (2018)

Title: Design and Implementation of Laboratory Fault Reporting System Based on Asp.net

Zetian Guo and Yingyue Chen presented an early but important work on digital fault reporting specifically for laboratory environments, built on the ASP.NET platform. Their system replaced phone-based reporting with a structured online form, demonstrated that growth in laboratory equipment directly correlates with increased maintenance complexity, and showed that even a basic web platform significantly reduces reporting delays.

Key Findings:

- Growth in laboratory equipment has increased fault occurrences and maintenance complexity.
- Traditional fault reporting methods such as phone reporting are inefficient and prone to delays.
- The system introduced an online platform for faster and more structured fault reporting.

Limitations:

The system focused narrowly on fault reporting alone, with no real-time status tracking, no mobile access, no automated technician assignment, and no administrative analytics. The platform was also limited to desktop web access and lacked any QR-code integration.

2.1.2 Giridharan, Nivedha, Vinothini, Maha, and Nivetha Kumari (2020)

Title: Web-Based Computer Maintenance Management System

The authors proposed a Web-Based Computer Maintenance Management System (CMMS) that centralized equipment data and maintenance records. Their key findings were that centralized data eliminates duplicated manual effort and that online complaint reporting with tracking significantly reduces troubleshooting time. The system demonstrated how a web-based approach reduces errors caused by manual data entry.

Key Findings:

- Centralized lab data and maintenance records lead to efficient maintenance management.
- Web-based CMMS significantly reduces manual work and data entry errors.
- Online complaint reporting with tracking enables faster troubleshooting and resolution.

Limitations:

The system lacked mobile access, automated technician assignment, and any predictive or analytical features. The absence of a mobile interface limited its utility for users who encounter faults while working in laboratory settings away from desktop computers.

2.1.3 Shakir A. Madiyev (2019)

Title: Computer System Maintenance in a Corporate Environment

Madiyev provided a comprehensive analysis of computer system maintenance strategies in corporate settings, covering reactive, preventive, predictive, and proactive approaches. The paper established that tracking maintenance indicators and service quality metrics leads to measurably better IT operations, and that condition monitoring enables early fault detection before failures become critical.

Key Findings:

- Maintenance indicators and service quality metrics improve IT operations performance.
- Multiple maintenance strategies including reactive, preventive, predictive, and proactive approaches enhance system reliability.

- Condition monitoring and diagnostics support early fault detection before critical failures occur.

Limitations:

While this work is theoretical rather than a working system implementation, it provided the conceptual foundation for the maintenance strategy framework incorporated into the proposed system. It does not address web or mobile interfaces, automated assignment, or laboratory-specific requirements.

2.1.4 Abiraj P, Jaiprasanth S.K, and Gunalan K (2021)

Title: Computer System Complaint Tracking System - Web Application

The authors developed a web-based complaint tracking system that introduced role-based workflows and real-time tracking. Their system showed that centralized tracking reduces resolution time and that analytics on fault trends enable data-driven decisions about spare parts and maintenance scheduling. The inclusion of a feedback mechanism after resolution was a notable feature that improved service quality accountability.

Key Findings:

- Analytics on fault trends and spare parts support data-driven maintenance decisions.
- Centralized, real-time complaint tracking enables faster resolution.
- Role-based workflow with feedback mechanisms improves service quality and accountability.

Limitations:

The system lacked a mobile application and any form of automated fault assignment. This meant that administrators still needed to manually assign technicians, which introduced delays and inefficiencies that the system was otherwise designed to eliminate.

2.1.5 Ramachandra Thirupathi, Ranganatha Rao, and T.R. Srinivas (2017)

Title: Design, Development and Implementation of Computerized Maintenance Management Information System (CMMIS)

The authors implemented a CMMIS that integrated equipment records, work orders, and inventory management into a single platform. Their work demonstrated that integrated maintenance modules streamline operations significantly and that real-time monitoring combined with historical tracking supports preventive maintenance decisions.

Key Findings:

- Integrated maintenance modules covering equipment, work orders, and inventory streamline maintenance operations.
- Real-time monitoring with historical data tracking enables effective preventive maintenance.
- Reduced downtime and improved resource utilization lead to higher efficiency and cost savings.

Limitations:

The system lacked a mobile interface and automated notification mechanisms. Its deployment was oriented toward corporate IT environments rather than educational laboratory settings, limiting its direct applicability to the institutional context of this project.

2.1.6 R. Razali, K.N. Abd Halim, and K. Jusoff (2009)

Title: Computer Management System

The authors examined computer management systems from an institutional governance perspective, focusing on service quality improvement and customer satisfaction. Their findings highlighted that management-driven approaches improve coordination across departments and that a focus on user satisfaction leads to better outcomes in service delivery.

Key Findings:

- Service quality improvement strategies enhance institutional performance.
- Management-driven approaches result in better coordination and control across departments.
- Focus on customer satisfaction leads to improved service delivery outcomes.

Limitations:

The paper did not address technical implementation details or specific fault management features. It provided useful insights into the administrative and governance aspects of maintenance systems but could not serve as a technical reference for system design.

2.2 Comparative Analysis

Table 1.1 presents a feature-level comparison of the six reviewed works against the proposed system across nine key parameters. This analysis confirms the novelty and comprehensiveness of the proposed integrated approach.

Paper	Fault Report	RT Track	Role-Based	Notif.	Admin Dash	Data Anal.	Mobile	Web	Auto Assign
Guo & Chen [1]	Yes	Partial	No	No	Partial	No	No	Yes	No
Giridharan et al. [2]	Yes	Yes	Partial	Partial	Yes	No	No	Yes	No
Madiyev [3]	Partial	Partial	No	No	Partial	Partial	No	No	No
Abiraj et al. [4]	Yes	Yes	Yes	Partial	Yes	Yes	No	Yes	No
Ramachandra et al. [5]	Yes	Yes	Partial	Partial	Yes	Yes	No	Partial	No
Razali et al. [6]	Partial	No	Partial	No	Yes	Partial	No	Partial	No
Proposed System	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Table 1.1: Comparison of Related Works vs Proposed System

2.3 Research Gap

Analyzing all six papers together, a consistent pattern emerges. Each system addresses one or two aspects of the problem effectively, but none of them provides a complete, integrated solution for the laboratory fault management context. The following specific gaps were identified:

- Only one paper (Guo and Chen [1]) focuses specifically on laboratory fault reporting. The remaining five papers address general computer maintenance or complaint tracking systems that are not tailored to the unique requirements of educational laboratory environments.

- None of the six systems combine web and mobile access in a single integrated solution. Mobile access is essential for students and faculty who encounter faults while working in laboratory settings.
- None of the reviewed systems include automated technician assignment based on workload, availability, and specialization. Manual assignment remains a source of delay and inefficiency in all reviewed works.
- None of the systems include QR-code based equipment identification as an integrated feature. Manual equipment identification introduces errors and slows the reporting process.
- None of the systems include predictive fault suggestions or cluster fault reporting mechanisms that group similar faults from multiple users.
- The absence of a comprehensive analytics and reporting module in most reviewed systems leaves administrators without the data-driven insights needed for preventive maintenance planning.

The proposed system was designed specifically to fill all of these gaps, delivering the first integrated laboratory fault management platform that satisfies all nine evaluation criteria simultaneously.

Sl. No.	Paper / Author & Year	System Title / Type	Key Findings / Advantages	Limitations / Disadvantages
1	Zetian Guo & Yingyue Chen (2018)	Design and Implementation of Laboratory Fault Reporting System Based on Asp.net	- Online structured form decreases reporting delays. - Successfully replaces completely manual paper records.	- Focuses only on basic submission. - Lacks real-time status tracking, QR integration, mobile accessibility, and automated technician assignment.
2	Giridharan et al. (2020)	Web-Based Computer Maintenance Management System	- Centralized lab records lead to organized maintenance workflows. - Reduces manual data entry errors significantly.	- Interface is restricted to desktop web environments. - Missing automated workloads, priority algorithms, and analytical dashboards.
3	Shakir A. Madiyev (2019)	Computer System Maintenance in a Corporate Environment	- Establishes that tracking service quality metrics improves IT operations. - Outlines condition monitoring methods.	- Theoretical architecture rather than a deployed framework. - Not tailored for multi-user academic lab structures.
4	Abiraj P, Jaiprasanth S.K, & Gunalan K (2021)	Computer System Complaint Tracking System - Web Application	- Role-based user workflows with feedback mechanisms. - Real-time complaint tracking improves general accountability.	- Lacks native mobile application support. - Assignment of technician jobs must still be executed manually by an administrator.
5	Ramachandra Thirupathi, Ranganatha Rao, & T.R. Srinivas (2017)	Computerized Maintenance Management Information System (CMMIS)	- Integrated functional modules (work orders, inventory). - Real-time monitoring supports basic preventive routines.	- Designed strictly for commercial corporate IT settings. - Lacks rapid on-site scanning entry tools (like QR codes) and push notifications.
6	R. Razali, K.N. Abd Halim, & K. Jusoff (2009)	Computer Management System	Proves that structured service quality frameworks improve departmental coordination.	- Purely focused on high-level administrative governance policy. - Lacks software engineering implementation details or architectural designs.
7	Proposed System (2026)	Smart Laboratory Equipment Fault Management System	- Full web/mobile cross-platform ecosystem. - Instant QR code system tracking. - Automated technician assignment and workload routing. - Live interactive administrative analytics.	System performance relies on stable internet connectivity for instant real-time cloud data synchronization.

Table 1.2: Literature Survey Summary

CHAPTER 3

SYSTEM DESIGN

3.1 System Overview

The Smart Laboratory Equipment Fault Management System is built around two primary interfaces that communicate through a shared backend. The web application serves administrators and technicians, providing full access to the dashboard, assignment tools, reporting engine, and system configuration. The mobile interface serves students and faculty, providing fault reporting, status tracking, and notification features. The backend is implemented with cloud-based services offering PostgreSQL database services, REST API generation, authentication, and cloud storage. All interfaces stay synchronized in real time through the shared API layer.

The system architecture is designed using a modular client-server approach to ensure scalability, maintainability, reliability, and efficient operational management. The architecture integrates frontend interfaces, backend services, database systems, authentication mechanisms, QR-based workflows, and analytics modules into a unified infrastructure management platform.

3.2 User Roles and Access Control

The system implements role-based access control with three distinct operational roles. Each role has a tailored interface and a defined set of permissions that ensures operational efficiency while maintaining data security.

Role 1: Students and Faculty (Fault Reporters)

- Scan equipment QR codes to access the fault reporting interface.
- Submit fault reports with issue title, description, category, priority, and supporting photographs.
- Monitor the real-time status of submitted fault reports.
- Receive push notifications when status changes occur.
- Access personal fault history with final resolution notes.

Role 2: Technicians (Maintenance Personnel)

- Receive assigned fault reports through the system dashboard.
- View full fault details including equipment information, location, and user description.
- Update the operational status of assigned faults (In Progress, Resolved).
- Add maintenance notes and photographs documenting the repair process.
- Mark faults as resolved and close issue records with maintenance history entries.

Role 3: Administrators (Oversight and Management)

- Access the full administrative dashboard with centralized visibility across all complaints and systems.
- Assign and reassign faults between technicians based on workload and specialization.
- Monitor complaint statistics, system health indicators, and technician performance metrics.
- Generate analytical reports filtered by date range, equipment category, technician, or laboratory.
- Manage equipment records, user accounts, QR codes, and system configuration settings.

3.3 Use Case Workflow

The operational workflow of the proposed system is designed to simplify and automate the process of reporting, tracking, assigning, and resolving infrastructure-related complaints within educational institutions. The workflow ensures that every complaint submitted by a user is properly recorded, monitored, and managed through a centralized platform.

3.3.1 Complete System Workflow

1. Admin configures labs and systems in the platform and generates QR codes for each equipment item.
2. QR code stickers are physically attached to the corresponding equipment.
3. User scans the QR code using a mobile device.

4. System automatically identifies the equipment and opens the issue reporting interface.
5. User submits the fault report with all required details.
6. Issue is stored in the centralized database and the admin receives a notification.
7. Admin reviews the issue, checks priority, and assigns a technician.
8. Technician receives the assignment notification and visits the lab.
9. Technician repairs or diagnoses the problem and updates the status.
10. Issue is marked as Resolved and maintenance history is saved.

3.3.2 Student / User Flow

The user flow represents the experience of a student or faculty member who encounters a fault:

- Open the application on the mobile device.
- Scan the QR code affixed to the faulty equipment.
- System details are automatically retrieved from the database.
- Issue reporting form opens with equipment details pre-populated.
- User enters the issue title, description, priority, and optionally uploads an image.
- User submits the issue report.
- Issue is stored in the database and the admin receives a notification.
- User receives a confirmation and can track the status in real time.

3.3.3 Admin Flow

The admin flow represents the experience of an administrator managing reported faults:

- Admin logs into the system using authenticated credentials.
- Administrative dashboard opens with a centralized overview.
- Admin views total labs, total systems, active issues, and pending repairs.
- Admin opens a reported issue to check full details.
- Admin assigns the issue to an appropriate technician.
- Admin updates issue status and monitors maintenance progress.
- User receives confirmation when the issue is resolved.

3.3.4 Technician Flow

The technician flow represents the experience of a maintenance personnel member:

- Technician logs into the system with assigned credentials.
- Technician views all issues assigned to their account.
- Technician opens the issue details to understand the problem.
- Technician visits the laboratory and inspects the affected system.
- Technician repairs or diagnoses the problem.
- Technician updates the status to In Progress and then to Resolved.
- Technician adds maintenance notes and closes the issue after resolution.

3.3.5 QR Code Internal Flow

The QR code integration operates as follows when a user scans a code:

- QR code contains either a system identifier or a unique system URL.
- Backend identifies the specific computer or equipment associated with the code.
- System data including laboratory name, system number, and maintenance history is fetched.
- Issue reporting form opens pre-populated with the identified system's information.

3.4 System Architecture

The architecture of the proposed system uses a modular client-server approach. The architecture integrates frontend interfaces, backend services, database systems, authentication mechanisms, QR-based workflows, and analytics modules into a unified management platform.

3.4.1 Architecture Overview

The system follows a cloud-based web architecture where the frontend application communicates with backend database services through secure APIs. The workflow begins when a user scans a QR code attached to equipment. The frontend application retrieves the associated system information and allows the user to submit complaints directly. The complaint data is then transmitted to the backend where it is securely stored in the centralized relational database.

3.4.2 Client-Side Architecture

The client-side architecture handles all user interactions and frontend rendering. The frontend is developed using React.js with TypeScript support for improved component reusability and maintainability. The component-based architecture organizes different operational modules into reusable interface components.

Primary Frontend Modules:

- Login and Authentication Interface
- Administrative Dashboard Interface
- Complaint Management Module
- Systems Monitoring Page
- Laboratory Management Page
- Analytics Dashboard
- QR-Based Complaint Registration Page

React Router is used for page navigation and protected route handling. Authentication validation mechanisms ensure that unauthorized users cannot access administrative functionalities. Responsive design principles are implemented throughout to ensure compatibility across desktops, tablets, and mobile devices.

3.4.3 Server-Side Architecture

The server-side architecture manages backend operations, database communication, authentication services, and centralized infrastructure data management. The backend infrastructure is powered using Supabase, providing PostgreSQL database services, REST API generation, cloud storage, authentication services, and session management.

The backend handles complaint storage, retrieval, technician assignment, authentication validation, and relational data management. It manages relationships between systems, locations, staff members, and complaints to ensure data consistency throughout the institution.

3.4.4 Deployment Architecture

The deployment architecture uses cloud-based hosting to ensure accessibility, scalability, and simplified operational management. The frontend application is deployed using Vercel,

which provides cloud hosting, automated deployment, high-speed content delivery, and scalable frontend infrastructure. Backend services and database operations are hosted using Supabase cloud infrastructure.

3.5 Database Design

The database architecture follows a relational model to ensure proper organization and association of infrastructure-related data. PostgreSQL is used as the primary database management system for its scalability, reliability, and support for complex relational queries.

3.5.1 Users Table

Column Name	Data Type	Description
user_id	INT (PK)	Unique identifier for every user
full_name	VARCHAR(100)	Name of the user
email	VARCHAR(100)	User email address
password	VARCHAR(255)	Encrypted password
role	ENUM	User role: admin, technician, staff, student
phone_number	VARCHAR(15)	Contact number
profile_image	VARCHAR(255)	User profile photo path
created_at	TIMESTAMP	Account creation date and time
updated_at	TIMESTAMP	Last updated timestamp

Table 3.1: Users Table Schema

3.5.2 Labs Table

Column Name	Data Type	Description
lab_id	INT (PK)	Unique identifier for each laboratory
lab_name	VARCHAR(100)	Name of the laboratory
block_name	VARCHAR(100)	College block or building name
floor_number	VARCHAR(10)	Floor where the lab is located
department	VARCHAR(100)	Department owning the laboratory
total_systems	INT	Number of systems in the lab
lab_incharge	VARCHAR(100)	Name of lab in-charge
created_at	TIMESTAMP	Lab creation timestamp

Table 3.2: Labs Table Schema

3.5.3 Systems Table

Column Name	Data Type	Description
system_id	INT (PK)	Unique identifier for each system
lab_id	INT (FK)	Connected laboratory identifier
system_number	VARCHAR(50)	Computer number used in the lab
qr_code	VARCHAR(255)	QR code linked to the system
processor	VARCHAR(100)	Processor details
ram	VARCHAR(50)	RAM size
storage	VARCHAR(100)	Storage capacity
operating_system	VARCHAR(100)	Installed operating system
system_status	ENUM	Status: working, issue_reported, under_repair, inactive
purchase_date	DATE	Date of purchase
last_service_date	DATE	Last maintenance date

Table 3.3: Systems Table Schema

3.5.4 Issue Reports Table

Column Name	Data Type	Description
issue_id	INT (PK)	Unique issue report identifier
system_id	INT (FK)	ID of the affected system
reported_by	INT (FK)	User who reported the issue
issue_title	VARCHAR(150)	Short title of the issue
issue_description	TEXT	Full description of the problem
issue_category	ENUM	Type: hardware, software, network, power, other
issue_priority	ENUM	Priority: low, medium, high, critical
issue_image	VARCHAR(255)	Uploaded image of the issue
issue_status	ENUM	Status: pending, assigned, in_progress, resolved, closed
reported_at	TIMESTAMP	Time when issue was reported
resolved_at	TIMESTAMP	Time when issue was resolved

Table 3.4: Issue Reports Table Schema

3.5.5 Technician Assignments Table

Column Name	Data Type	Description
assignment_id	INT (PK)	Unique assignment identifier
issue_id	INT (FK)	Related issue identifier
technician_id	INT (FK)	Assigned technician

Column Name	Data Type	Description
assigned_by	INT (FK)	Admin who assigned the issue
assigned_date	TIMESTAMP	Date of assignment
assignment_status	ENUM	Status: assigned, accepted, completed

Table 3.5: Technician Assignments Table Schema

3.5.6 Foreign Key Relationships

Parent Table	Child Table	Relationship
Labs	Systems	One-to-Many
Systems	Issue Reports	One-to-Many
Users	Issue Reports	One-to-Many
Users	Technician Assignments	One-to-Many
Issue Reports	Technician Assignments	One-to-One
Systems	Maintenance History	One-to-Many
Users	Notifications	One-to-Many
Users	Activity Logs	One-to-Many

Table 3.6: Foreign Key Relationships

Table 3.7: Notifications Table - Database Schema

1. NOTIFICATIONS TABLE - DATABASE SCHEMA

Column Name	Data Type	Constraints / Key	Description
notification_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique notification ID
user_id	INT	FOREIGN KEY (users.user_id)	User who receives the notification
notification_title	VARCHAR(150)	NOT NULL	Notification heading / title
notification_message	TEXT	NOT NULL	Detailed notification message
notification_type	ENUM('issue','assignment','status_update','reminder')	NOT NULL	Type of notification
is_read	BOOLEAN	DEFAULT FALSE	Read status of the notification
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the notification was created

2. ACTIVITY LOGS TABLE - DATABASE SCHEMA

Column Name	Data Type	Constraints / Key	Description
log_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique log ID
user_id	INT	FOREIGN KEY (users.user_id)	User who performed the activity
activity	TEXT	NOT NULL	Description of the action performed
ip_address	VARCHAR(50)	NULL	IP address of the user
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time when the activity occurred

Table 3.8: Activity Logs Table - Database Schema

Complete System Workflow Diagram

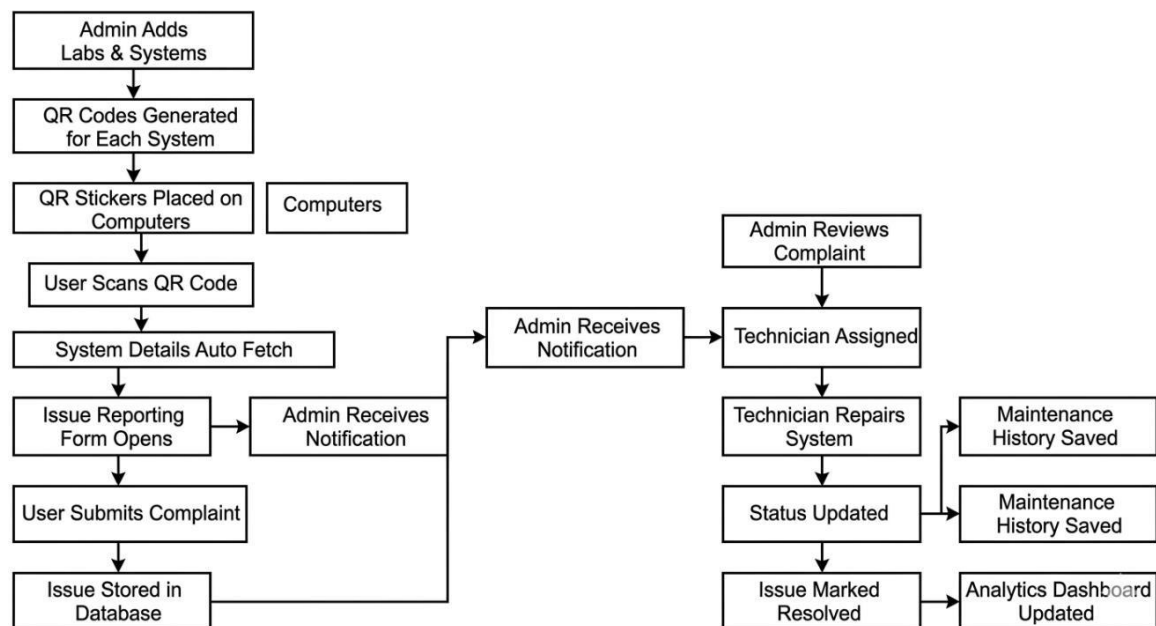


Figure 3.1

User Flow – QR Based Lab Issue Tracking

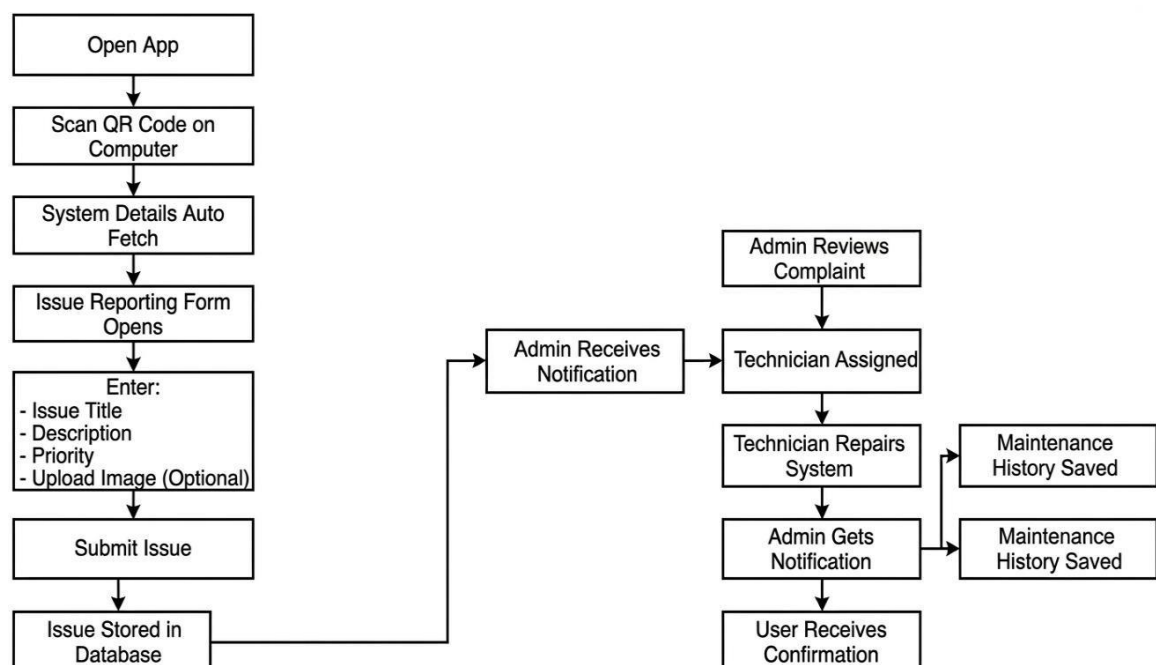


Figure 3.2

Admin Flow – Dashboard and Issue Management

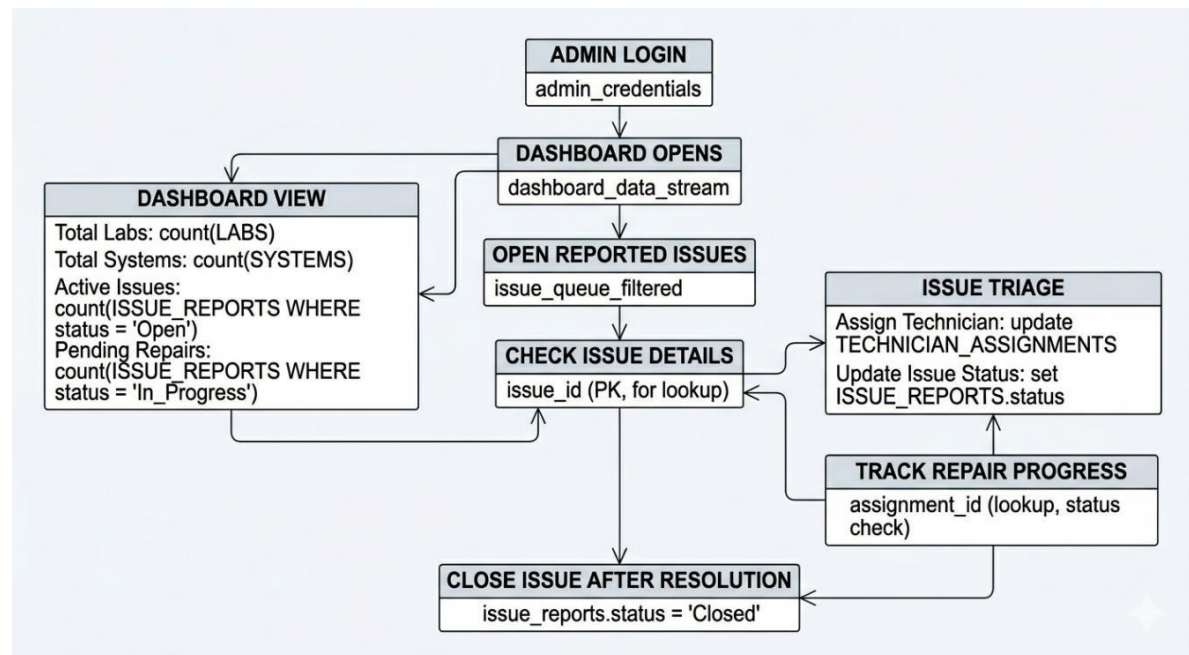


Figure 3.3

Technician Flow

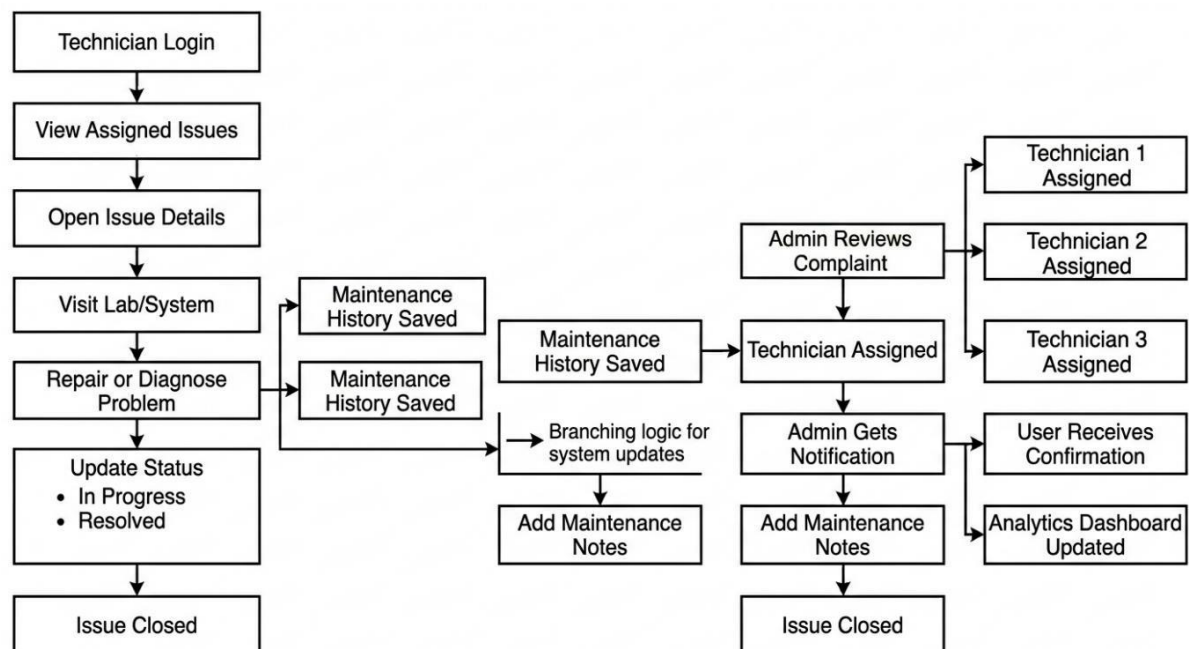


Figure 3.4

QR Code Scanning Internal Flow

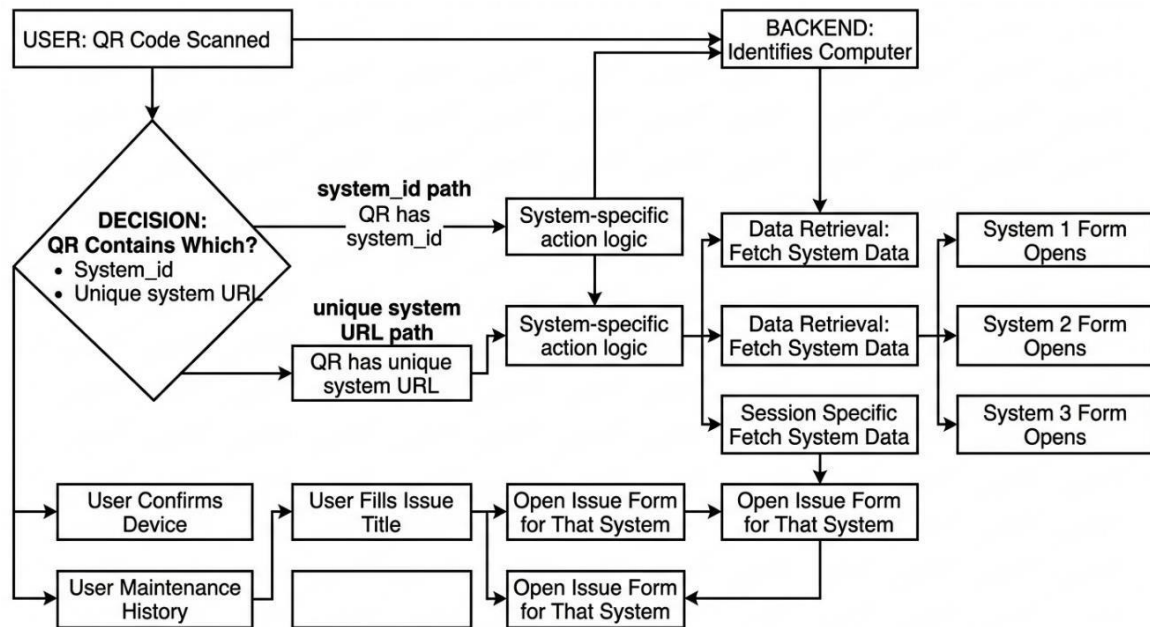


Figure 3.5

System Architecture Overview

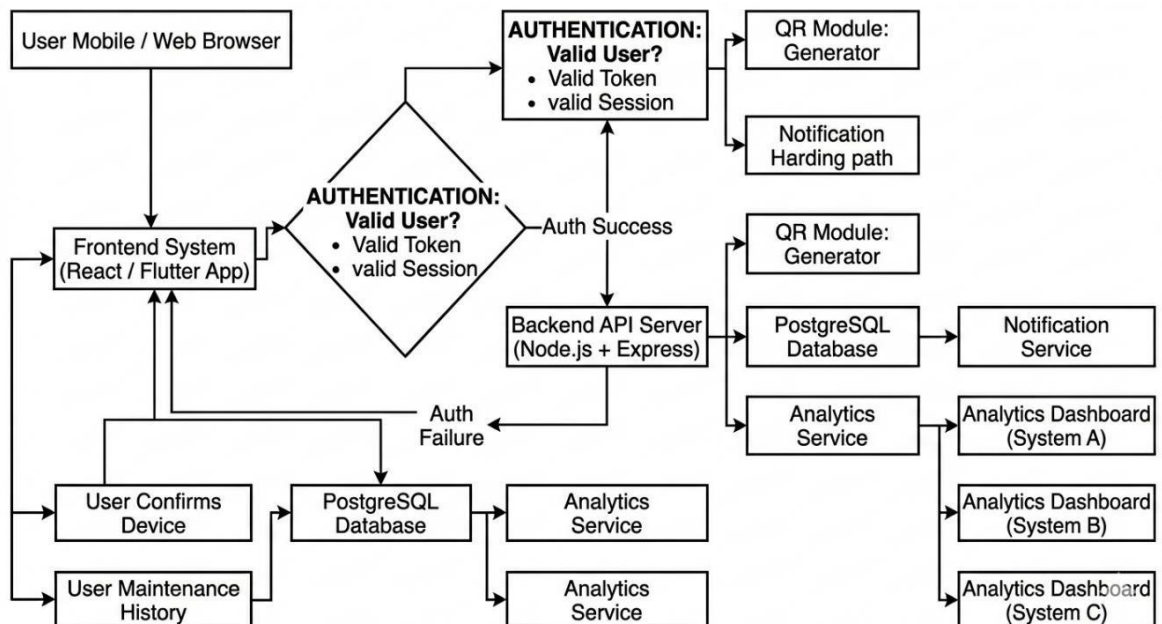


Figure 3.6

Client-Server Architecture Diagram

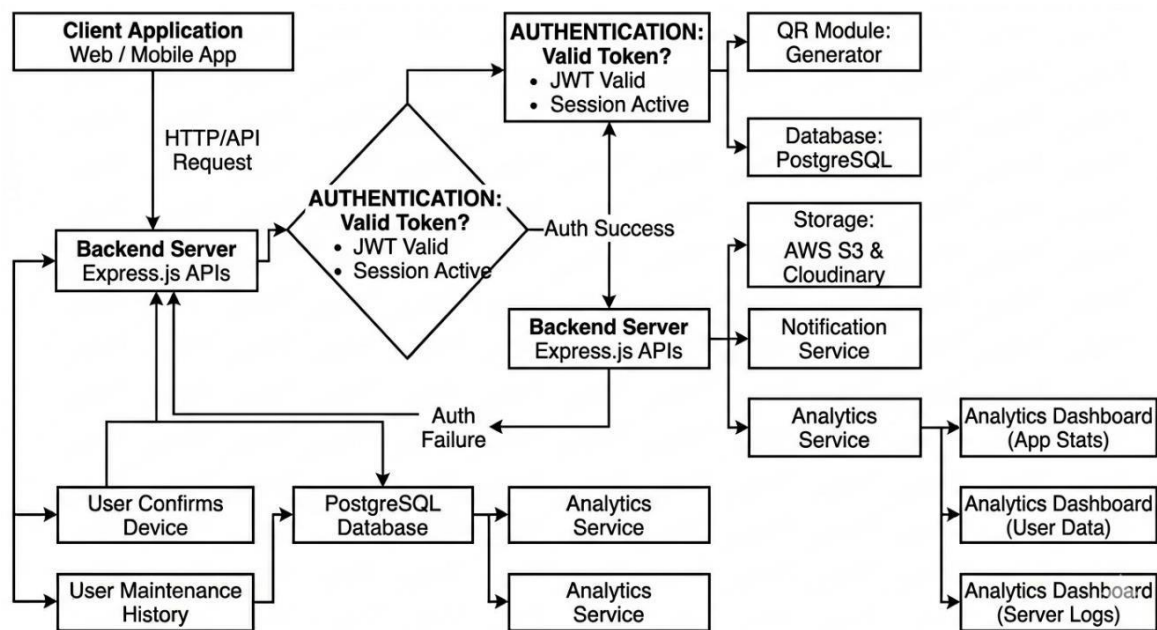


Figure 3.7

ER Diagram – Database Relationships

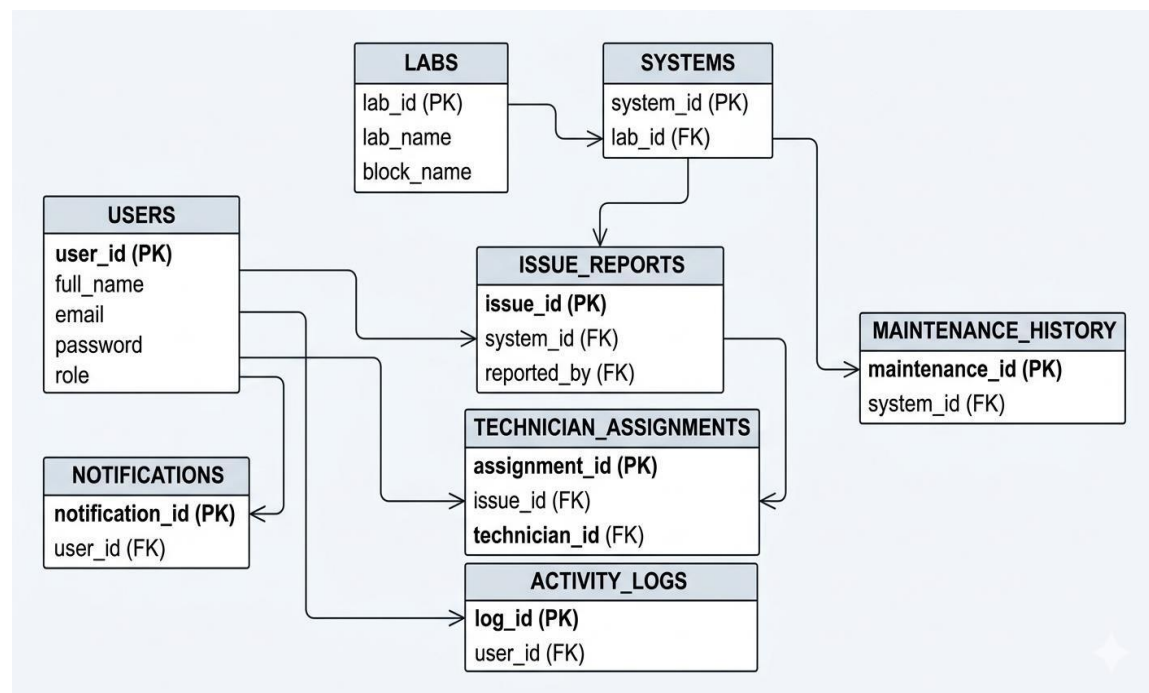


Figure 3.8

CHAPTER 4

METHODOLOGY AND IMPLEMENTATION

The methodology of the Smart Laboratory Equipment Fault Management System encompasses the systematic design principles, implementation strategies, and operational results that transform the proposed architecture into a fully functional platform. The methodology follows a modular development approach where individual components are developed and integrated progressively.

4.1 Important Parameters

Several important parameters were considered during the system design and implementation phases to ensure that the proposed platform remains reliable, scalable, secure, and user-friendly.

4.1.1 Scalability

Scalability refers to the capability of the system to efficiently manage increasing numbers of laboratories, computer systems, complaints, and administrative users without degrading overall system performance. Since educational institutions may continuously expand their infrastructure over time, the system architecture supports future growth with minimal structural modifications. The relational database design, modular frontend components, and cloud deployment architecture collectively ensure that the platform can accommodate additional resources and users while maintaining operational stability.

4.1.2 Performance

Performance is an essential parameter because the application is expected to process complaints, retrieve infrastructure data, and display administrative dashboards in real time. Efficient database queries, optimized frontend rendering, and cloud-based hosting technologies are utilized to reduce response times and improve user experience. The system provides quick complaint submission and fast retrieval of infrastructure-related information even when handling large volumes of data.

4.1.3 Security

Security plays a vital role in protecting administrative functionalities and maintaining the integrity of institutional data. The system implements authentication and protected routing mechanisms to ensure that only authorized personnel can access management features such as complaint assignment, infrastructure monitoring, and analytics dashboards. Public users are restricted to complaint registration functionalities only. Secure backend services and authenticated sessions help prevent unauthorized access and protect sensitive operational data.

4.1.4 Usability

Usability is one of the most important design parameters because the effectiveness of the system depends heavily on user interaction. The interface is designed to be simple, intuitive, and accessible to users with minimal technical knowledge. QR-based complaint reporting significantly reduces the complexity involved in identifying systems and registering complaints manually. The responsive design further improves accessibility by enabling the platform to function efficiently across desktops, tablets, and mobile devices.

4.1.5 Maintainability

Maintainability refers to the ease with which the system can be modified, updated, or extended in the future. The application follows a modular development architecture where frontend components, database entities, and operational modules are separated based on functionality. This approach improves code readability, debugging efficiency, and future feature integration. Additional functionalities such as notification systems, AI-based analytics, or mobile applications can be integrated into the existing architecture without requiring complete system redesign.

4.1.6 Reliability

Reliability ensures that the system consistently performs its intended operations without failures or interruptions. Since complaint management directly impacts institutional maintenance workflows, the application must remain stable and continuously accessible. Cloud hosting infrastructure and centralized backend services improve system availability and reduce operational downtime. Proper database relationships and validation mechanisms also ensure consistency and accuracy of data.

4.1.7 Accessibility

Accessibility refers to the ability of users and administrators to access the system from different locations and devices without operational limitations. The proposed application is deployed on cloud infrastructure, allowing authorized staff members to monitor complaints and infrastructure data remotely. Mobile-friendly interfaces and browser-based accessibility further improve convenience for users reporting complaints through QR-based workflows.

4.2 Architecture

The architecture of the proposed system is designed using a modular client-server approach to ensure scalability, maintainability, reliability, and efficient operational management. The architecture integrates frontend interfaces, backend services, database systems, authentication mechanisms, QR-based workflows, and analytics modules into a unified infrastructure management platform.

4.2.1 Client Side Architecture

The client-side architecture is responsible for handling all user interactions and frontend rendering. The frontend is developed using React.js with TypeScript to improve component reusability, maintainability, and application scalability. The architecture employs a component-based design in which different operational modules are separated into reusable user interface components.

The frontend application uses React Router for page navigation and protected route handling. Authentication validation mechanisms ensure that unauthorized users cannot access administrative functionalities. Responsive design principles are implemented throughout the application to ensure compatibility across desktops, tablets, and mobile devices.

4.2.2 Server Side Architecture

The server-side architecture is responsible for managing backend operations, database communication, authentication services, and centralized infrastructure data management.

The backend infrastructure is powered using Supabase, which provides PostgreSQL database services, REST API generation, cloud storage, authentication services, and session management.

The backend handles critical operations including complaint storage, complaint retrieval, technician assignment, authentication validation, infrastructure monitoring, and relational data management. The server-side architecture manages relationships between systems, locations, staff members, and complaints to ensure data consistency.

4.2.3 Database Architecture

The database architecture follows a relational database model to ensure proper organization and association of infrastructure-related data. PostgreSQL is used as the primary DBMS for its scalability, reliability, and support for complex relational queries. The database structure consists of several interconnected tables that collectively manage institutional infrastructure operations.

4.2.4 Authentication Architecture

Authentication mechanisms are implemented to protect administrative functionalities from unauthorized access. The authentication workflow includes username-based login, password validation, session authentication, and protected route authorization. Administrative pages such as the dashboard, analytics, complaints management, and systems management are accessible only after successful authentication. The QR-based complaint reporting interface remains publicly accessible because end users must be able to register complaints quickly without requiring login credentials.

4.2.5 QR Integration Architecture

The QR integration module is one of the most important architectural components. Each computer system is associated with a dynamically generated QR code linked to its unique system identifier. The QR code contains a URL structured as: `https://domain-name/report/system-id`. When a user scans the QR code, the browser automatically opens the complaint reporting interface. The frontend extracts the system identifier from URL parameters and retrieves the associated laboratory and system information from the database.

4.2.6 Analytics Architecture

The analytics architecture monitors operational data and generates infrastructure insights for administrators. The analytics module processes complaint and infrastructure information to display total complaints, pending issues, resolved complaints, laboratory statistics, technician performance metrics, and infrastructure health indicators. Real-time database queries ensure that dashboard analytics remain continuously updated.

4.3 Implementation

The implementation phase transforms the designed architecture into a fully functional infrastructure complaint management platform. The implementation process involves frontend development, backend integration, database configuration, authentication setup, QR-based complaint workflows, analytics generation, and cloud deployment.

4.3.1 Frontend Implementation

The frontend implementation is carried out using React.js with TypeScript support. The frontend architecture follows a component-based development methodology where reusable user interface components are organized based on operational functionalities. Major frontend pages implemented include the Dashboard Page, Complaint Management Page, Systems Management Page, Laboratory Management Page, Analytics Dashboard, Login Page, and QR Complaint Reporting Page.

The frontend implements dynamic rendering techniques where infrastructure information is fetched directly from the backend database and displayed in real time. Each page is connected through React Router to provide smooth navigation and protected route handling.

4.3.2 Authentication Implementation

The authentication module is implemented using Supabase Authentication services. The implemented authentication workflow includes username-based login, password validation, session persistence, protected route authorization, and logout functionality.

Protected routes restrict unauthorized access to administrative modules while public routes such as the QR complaint reporting page remain accessible without credentials.

4.3.3 Database Implementation

The database implementation is performed using Supabase PostgreSQL services following a relational design to maintain proper associations between infrastructure entities. The implementation includes the creation of multiple interconnected tables for laboratories and office locations, computer systems, complaint records, staff members, and complaint assignments. Foreign key relationships are implemented to establish data consistency between different operational modules.

4.3.4 QR-Based Complaint Registration Implementation

The QR-based complaint reporting module is a core implementation of the proposed system. Each computer system is associated with a unique QR code generated dynamically using the system identifier. When a user scans the QR code using a mobile device, the browser automatically opens the complaint reporting interface. The frontend extracts the system identifier from the URL parameters and retrieves the associated laboratory and system information. Users can then select issue type, enter complaint description, and submit infrastructure issues instantly.

4.3.5 Complaint Management Implementation

The complaint management module provides administrators and technicians with centralized operational control over infrastructure issues. The complaints page retrieves all complaint records from the backend and displays complaint details, system information, assigned staff, issue priority, and complaint status. Each complaint entry is an interactive workflow item accessible through a dedicated complaint detail page that enables technician assignment, status updates, and issue tracking.

4.3.6 Analytics Module Implementation

The analytics implementation generates operational insights from complaint and infrastructure data stored in the centralized database. The analytics module dynamically calculates total complaints, pending issues, resolved issues, resolution rates, technician

performance, and laboratory complaint distribution. Dynamic aggregation and filtering techniques process complaint data to generate infrastructure statistics in real time, helping administrators identify recurring problems and improve maintenance planning.

4.3.7 Deployment Implementation

The deployment implementation uses cloud-based infrastructure for application accessibility, scalability, and centralized operational management. The frontend is deployed using Vercel cloud hosting services with continuous deployment integrated with GitHub repositories. Backend database services and authentication infrastructure are deployed using Supabase cloud services. The cloud deployment ensures that the application remains accessible across multiple devices and locations.

4.4 Results and Discussion

The implementation of the proposed system successfully demonstrates the feasibility of a centralized QR-based infrastructure complaint management platform for educational institutions. The results obtained indicate significant improvements in complaint handling efficiency, infrastructure visibility, and maintenance workflow management compared to traditional manual complaint registration methods.

4.4.1 Dashboard Performance Results

The dashboard module successfully provides administrators with a centralized operational overview of institutional infrastructure conditions. Real-time complaint statistics, system health indicators, and maintenance summaries are dynamically retrieved from the backend database and displayed through interactive interface components. The dashboard enables administrators to monitor total complaints, pending maintenance requests, systems under maintenance, active laboratories, and recent operational activities. The real-time nature of the dashboard allows maintenance teams to identify critical issues immediately after complaint submission.

4.4.2 QR-Based Complaint Registration Results

The QR-based complaint registration workflow successfully reduces the complexity involved in reporting infrastructure-related issues. Users can register complaints directly

by scanning QR codes attached to systems without manually entering laboratory or system details. The QR integration workflow produced several operational advantages including faster complaint registration, reduced user effort, elimination of incorrect system identification, simplified reporting process, and improved accessibility through mobile devices. Complaints submitted through QR interfaces appear instantly within the administrative dashboard, demonstrating successful backend integration and real-time database synchronization.

4.4.3 Complaint Management Results

The complaint management module successfully centralizes all infrastructure complaints within a single operational interface. Administrators and technical staff can monitor complaint lifecycles, assign technicians, and update issue statuses dynamically. The implemented workflow enables complaints to transition through operational stages including Pending, In Progress, and Resolved. The technician assignment mechanism improves accountability by associating maintenance responsibilities with specific staff members and reduces dependency on verbal communication and manual tracking methods.

4.4.4 Laboratory Monitoring Results

The laboratory monitoring module effectively visualizes infrastructure conditions across laboratories and office environments. The system dynamically calculates laboratory health percentages based on the operational status of systems associated with each location. Administrators can identify unhealthy laboratories, recurring maintenance locations, infrastructure distribution, and maintenance-heavy environments. The visual representation through operational cards and system tiles improves infrastructure awareness and management efficiency.

4.4.5 Analytics Results

The analytics module successfully generates operational insights using complaint and infrastructure data. The analytics dashboard provides visibility into complaint distribution, issue categories, resolution rates, technician performance, and laboratory-wise complaint frequency. The implementation demonstrates the potential of analytics-driven infrastructure management within educational institutions. The collected operational data can further support future infrastructure planning and preventive maintenance strategies.

4.4.6 Authentication and Security Results

The authentication module successfully restricts administrative functionalities to authorized staff members only. Protected routing mechanisms prevent unauthorized access to operational dashboards and management modules. The implemented session-based authentication workflow ensures secure administrative access while maintaining public accessibility for QR-based complaint reporting. This hybrid security model improves operational flexibility by allowing public complaint registration without compromising administrative control over data.

4.4.7 Advantages of the Proposed System

No.	Advantage	Description
1	Centralized Management	All complaints and infrastructure data managed from a single platform
2	QR-Based Reporting	Instant and accurate equipment identification without manual data entry
3	Real-Time Monitoring	Live dashboard provides immediate visibility into infrastructure health
4	Technician Assignment	Structured workflow for assigning and tracking maintenance responsibilities
5	Operational Analytics	Data-driven insights for maintenance planning and performance evaluation
6	Reduced Manual Work	Automated workflows eliminate manual communication and tracking
7	Cloud Accessibility	Application accessible from any device and location with internet
8	Scalable Architecture	System supports growth without requiring structural redesign

Table 4.1: Advantages of the Proposed System

4.4.8 Limitations of the System

No.	Limitation	Description
1	No Mobile App	Dedicated native mobile application not yet implemented
2	No Push Notifications	Automated notification system not yet integrated in current version
3	No Predictive Maintenance	AI-based fault prediction not yet incorporated
4	Internet Dependency	System requires active internet connectivity for all operations
5	No Offline Mode	Complaint registration not possible without network connection
6	Manual QR Setup	QR codes must be manually generated and physically affixed to equipment

Table 4.2: Limitations of the System

Use Case Diagram

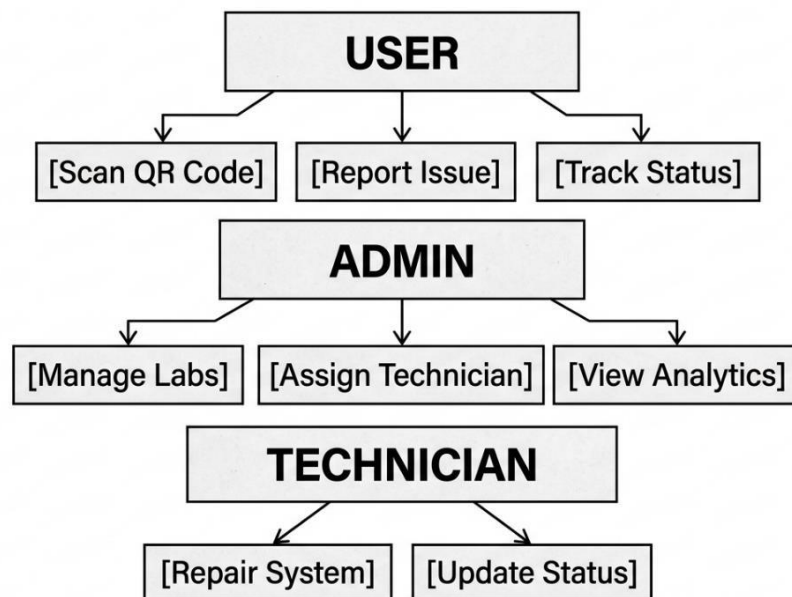


Figure 4.1

QR-Based Complaint Registration Flow

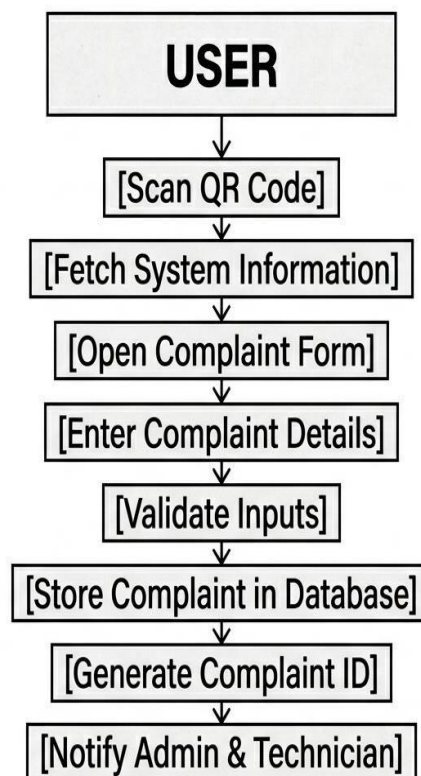


Figure 4.2

Complaint Lifecycle Status Flow

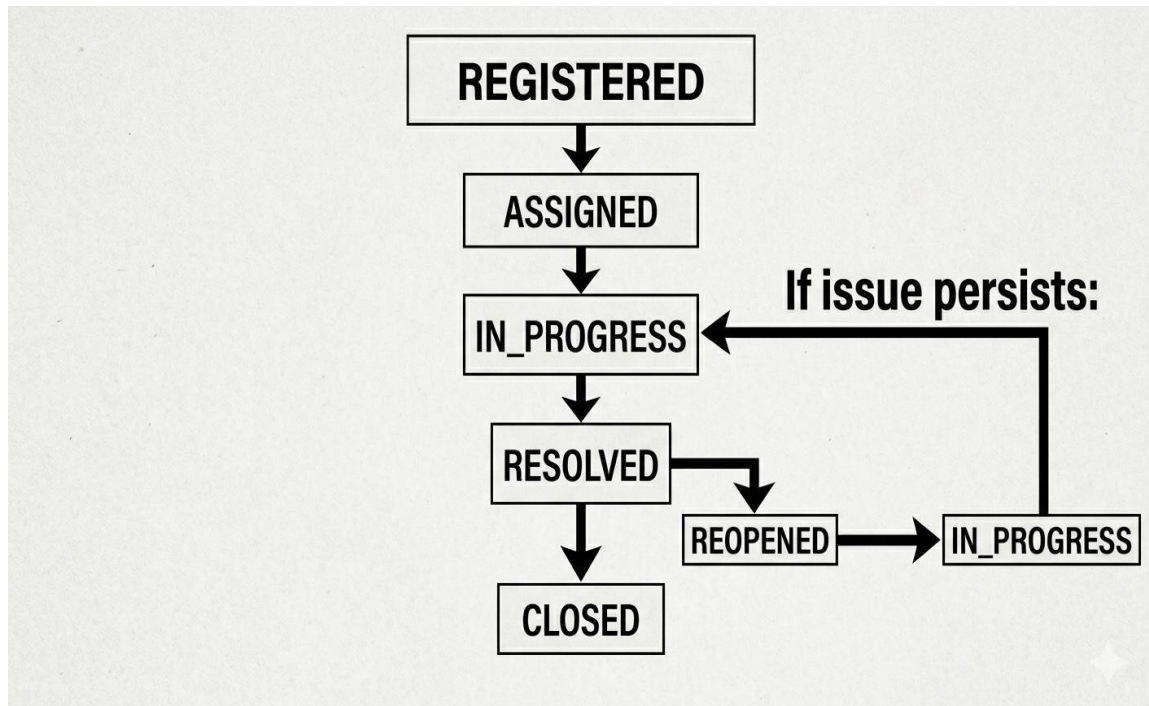
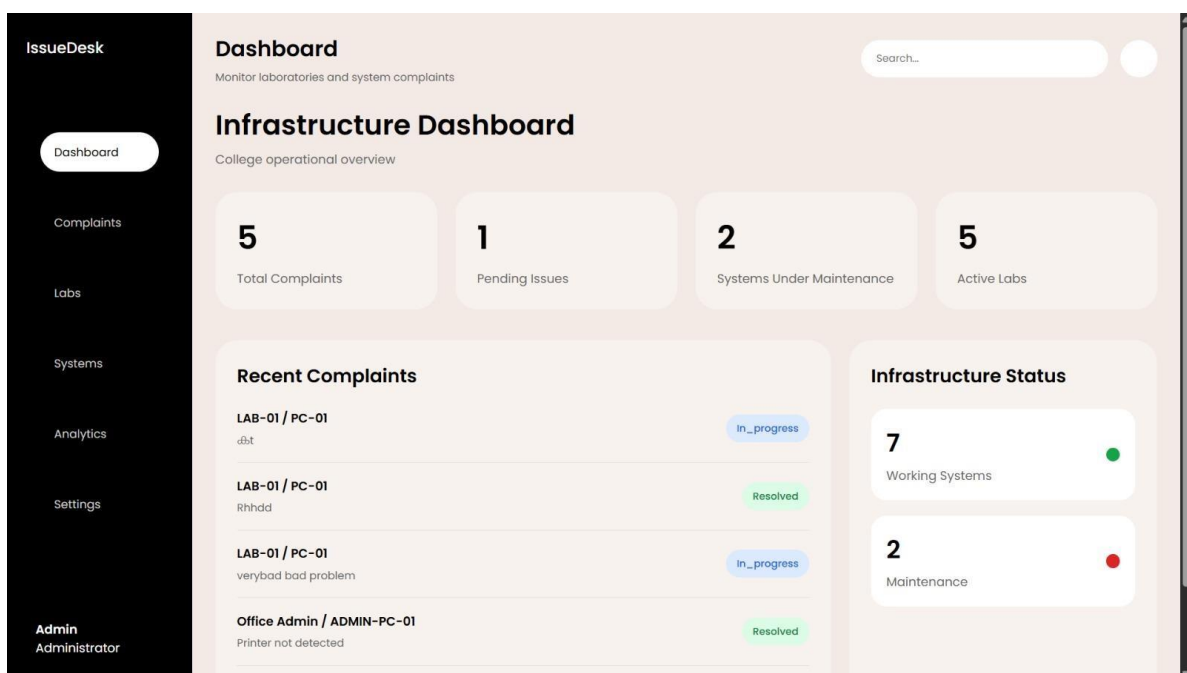
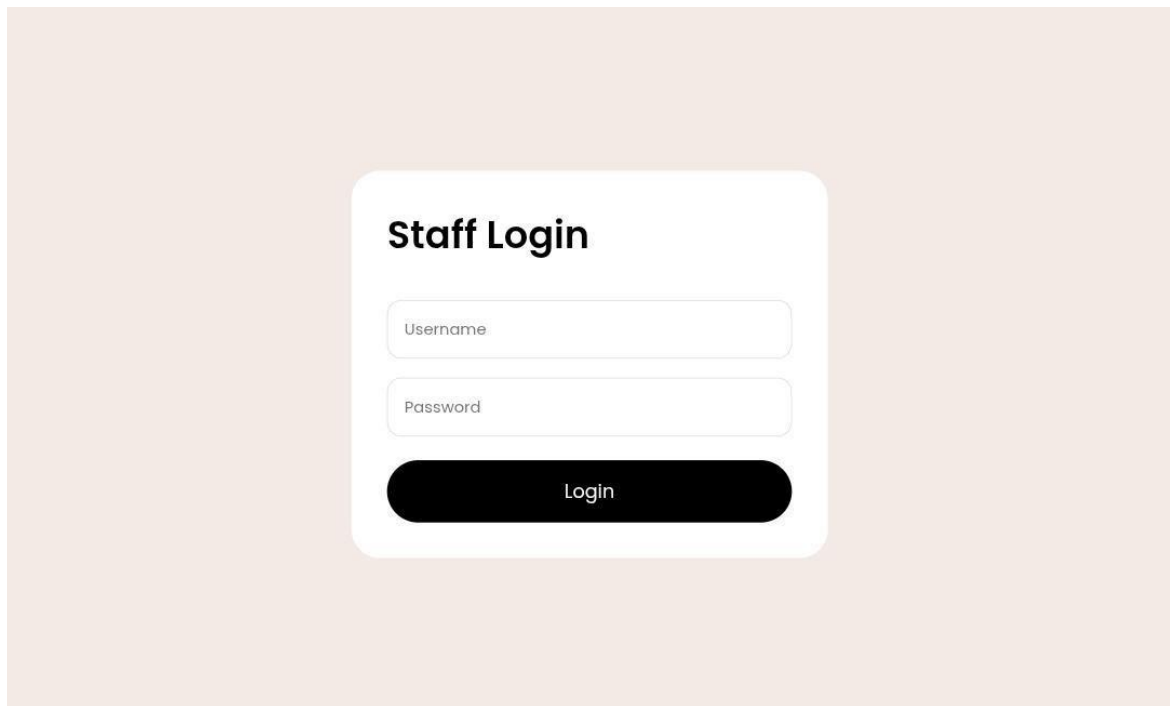


Figure 4.3

SYSTEM USER INTERFACE

Desktop Interface



IssueDesk

Dashboard

Complaints

Labs

Systems

Analytics

Settings

Admin
Administrator

Dashboard

Monitor laboratories and system complaints

Search...

Complaints

Manage all reported issues

Add Complaint

Lab / Office	Computer	Complaint	Assigned Staff	Priority	Status
Office Admin	ADMIN-PC-01	Printer not detected	Rahul	low	resolved
LAB-02	PC-01	Windows crashing repeatedly	Arjun	high	pending
LAB-01	PC-02	Keyboard keys not working	Rahul	medium	in_progress
LAB-01	PC-01	Monitor not turning on	Arjun	high	pending

IssueDesk

Dashboard

Complaints

Labs

Systems

Analytics

Settings

Admin
Administrator

Dashboard

Monitor laboratories and system complaints

Search...

Complaint #4

Office Admin / ADMIN-PC-01

resolved

Issue Description

Printer not detected

Assign Technician

Rahul

Update Status

Pending

In Progress

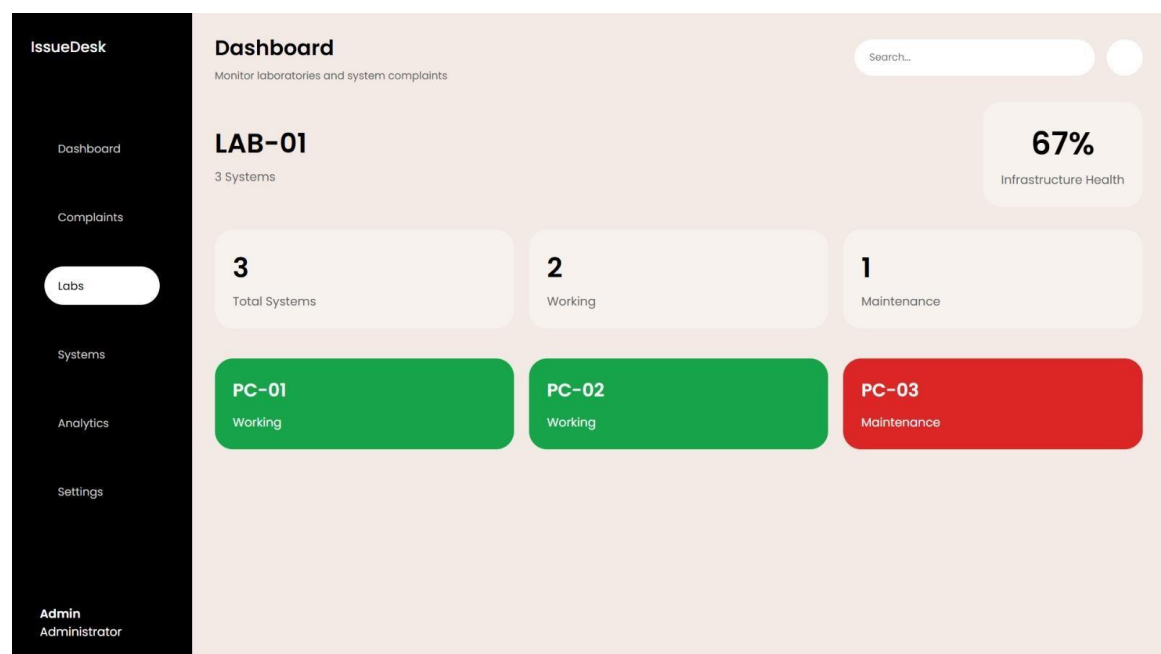
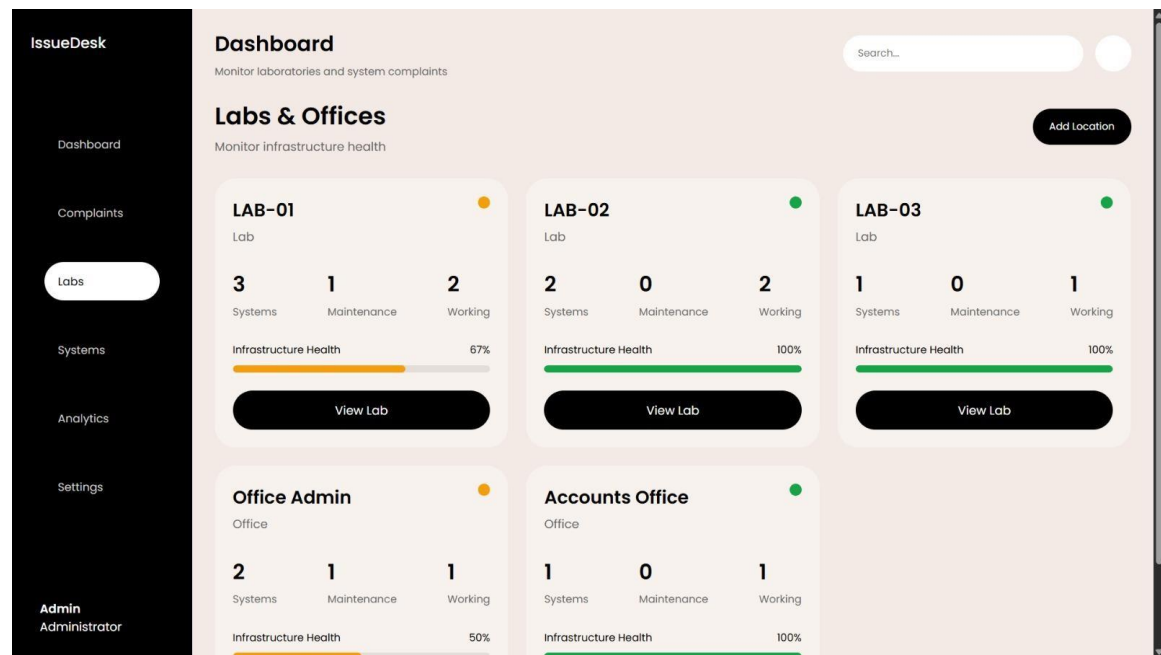
Resolved

Complaint Info

Priority: low

Issue Type: Peripheral

Assigned To: Rahul



IssueDesk

Dashboard

Complaints

Labs

Systems

Analytics

Settings

Admin Administrator

Dashboard

Monitor laboratories and system complaints

Search...

Systems

Manage all computers and QR access

Add System

ID	Computer	Location	Status	QR Access
#1	PC-01	LAB-01	working	Open QR Page
#2	PC-02	LAB-01	working	Open QR Page
#3	PC-03	LAB-01	maintenance	Open QR Page
#4	PC-01	LAB-02	working	Open QR Page
#5	PC-02	LAB-02	working	Open QR Page
#6	PC-01	LAB-03	working	Open QR Page
#7	ADMIN-PC-01	Office Admin	working	Open QR Page

IssueDesk

Dashboard

Complaints

Labs

Systems

Analytics

Settings

Admin Administrator

Dashboard

Monitor laboratories and system complaints

Search...

Analytics

Operational infrastructure insights

7

Total Complaints

2

Pending Issues

2

Resolved Issues

29%

Resolution Rate

Issue Categories

Hardware	2
Software	2
Peripheral	1
Network	2

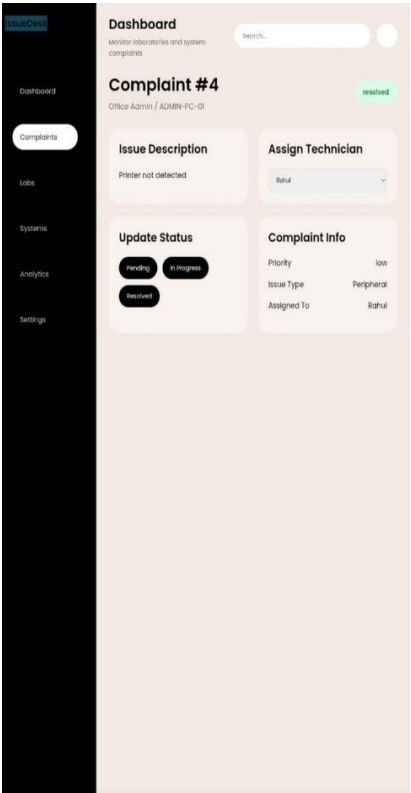
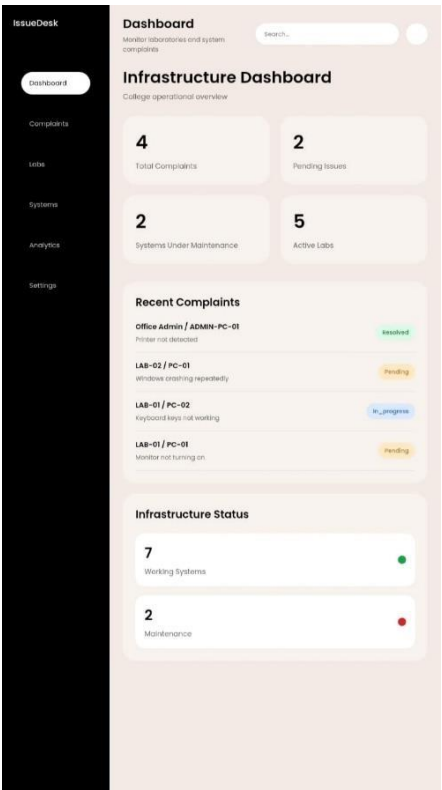
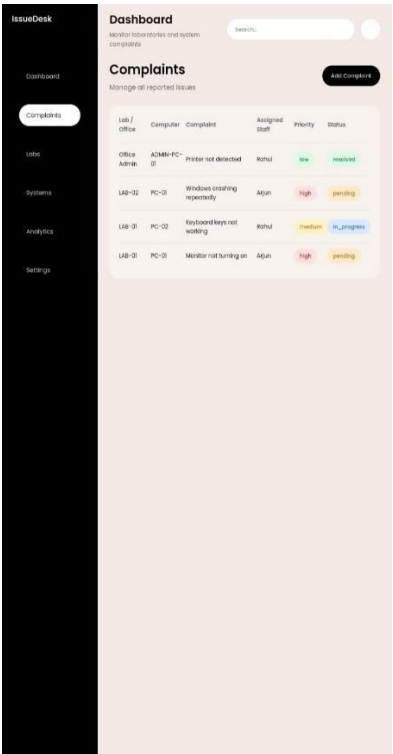
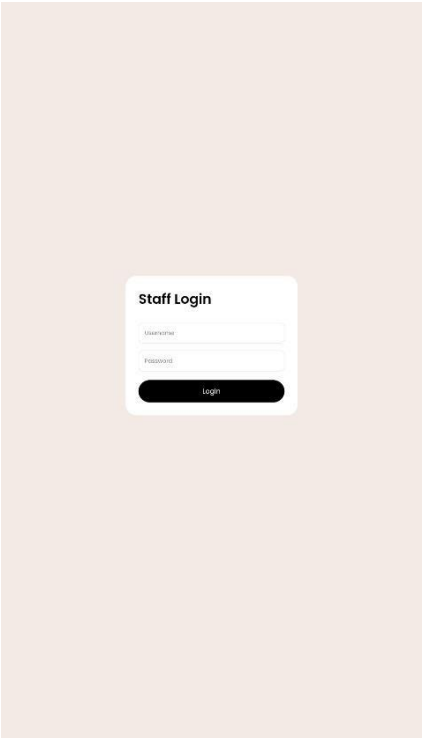
Most Problematic Labs

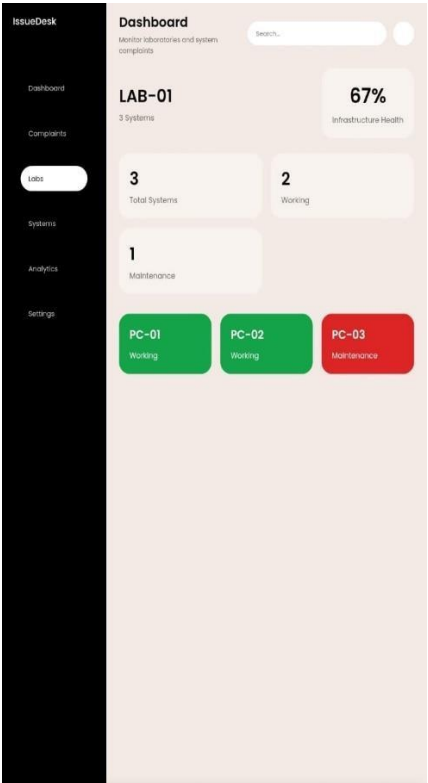
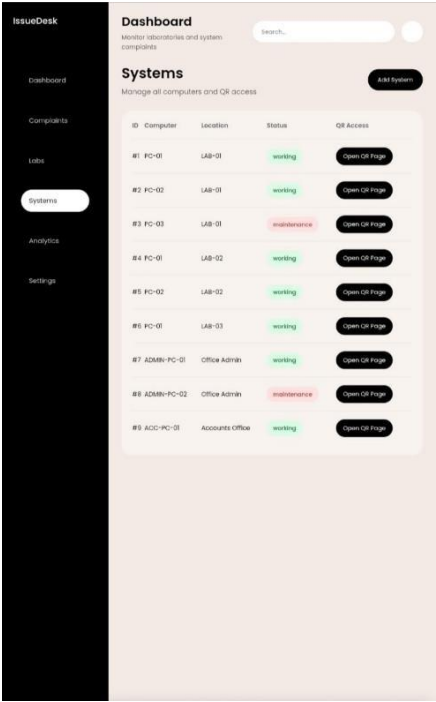
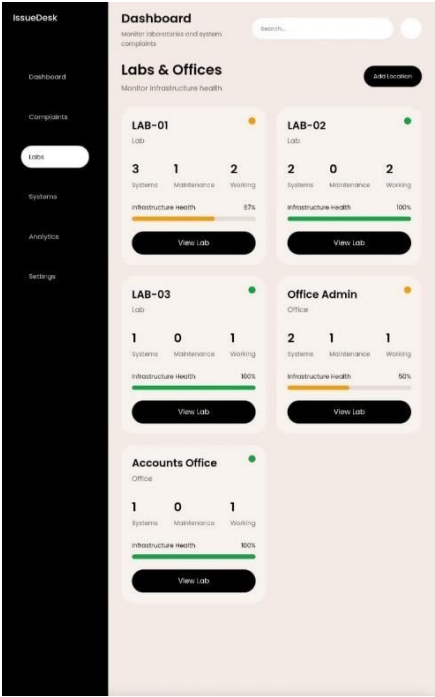
LAB-01	5
LAB-02	1
Office Admin	1

Technician Performance

Rahul	1 Resolved
Arjun	1 Resolved

Mobile Interface





Report Issue

Lab: LAB-01
System: PC-01

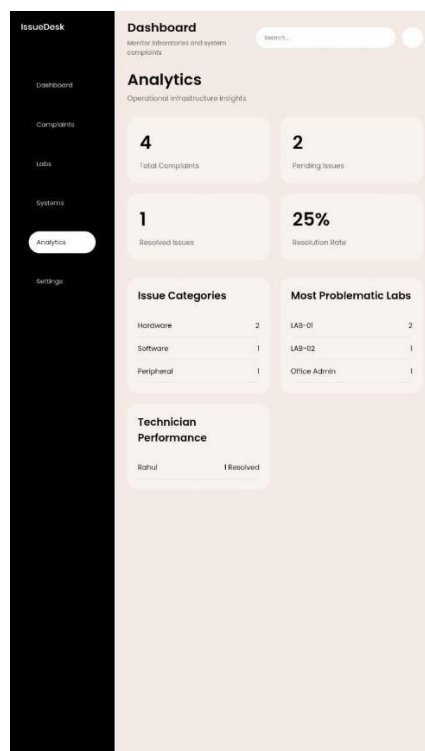
Issue Type

Hardware

Describe Issue

Describe the issue...

Submit Complaint



CHAPTER 5

PROPOSED SYSTEM FEATURES

The Smart Laboratory Equipment Fault Management System introduces a comprehensive set of features that address the research gaps identified in the literature survey. This chapter presents the key features, technology stack, and comparative advantages of the proposed system.

5.1 Key Features

5.1.1 QR-Code Based Equipment Identification

Every piece of laboratory equipment is assigned a unique QR code that encodes its system identifier and links to its specific complaint reporting interface. Scanning the code during fault reporting eliminates manual data entry errors and ensures that fault records are always linked to the correct equipment record in the database. The QR code URL follows the format: `https://domain-name/report/system-id`, which allows the backend to automatically identify the equipment, retrieve its information, and pre-populate the complaint form.

5.1.2 Automated Fault Prioritization

When a fault report is submitted, the system applies a priority auto-tagging algorithm that classifies the fault as low, medium, high, or critical priority based on the equipment type, the reported fault category, and the frequency of previous faults on that item. This automated classification ensures that critical infrastructure issues receive immediate attention while lower-priority issues are queued appropriately.

5.1.3 Cluster Fault Reporting

When multiple users report the same fault on the same equipment within a short time window, the system groups these into a single cluster report. This prevents technicians from receiving duplicate assignments and gives administrators an accurate picture of the severity and scope of the issue. Cluster detection reduces administrative workload and ensures that maintenance resources are focused on resolving the actual underlying problem rather than processing redundant reports.

5.1.4 Predictive Common Fix Suggestion

The system analyses historical fault resolution data for each piece of equipment and, when a new fault is reported, suggests the most common resolution action to the assigned technician. This reduces diagnostic time for recurring issues and serves as a lightweight AI-assisted maintenance advisory. As the system accumulates more fault history data, the accuracy and usefulness of these suggestions improve progressively.

5.1.5 Repeated Fault Detector

The system flags equipment that has been the subject of fault reports more than a configurable threshold number of times within a specified period. This alert helps administrators identify equipment that may need replacement rather than continued repair, enabling data-driven decisions about maintenance investment versus equipment procurement.

5.1.6 Real-Time Administrative Dashboard

The administrative dashboard provides centralized real-time visibility into all aspects of institutional infrastructure health. The dashboard displays total complaints, pending maintenance requests, systems under maintenance, active laboratories, recent operational activities, laboratory-wise issue distribution, technician workload, and resolution performance metrics. All data is dynamically retrieved from the backend database and refreshed in real time.

5.1.7 Automated Notifications

The system dispatches automated notifications to all relevant stakeholders at every stage of the fault resolution lifecycle. When a fault is submitted, the reporting user receives a confirmation notification. When the fault is assigned to a technician, the technician receives an assignment notification. As the technician updates the status, the reporting user receives status update notifications. When the fault is resolved, the user receives a resolution confirmation with maintenance notes.

5.1.8 Maintenance History Tracking

Every maintenance activity performed on every piece of equipment is recorded in the maintenance history table with details including the technician involved, the maintenance performed, any replaced parts, the maintenance cost, and the date of service. This creates a comprehensive audit trail that supports warranty management, equipment lifecycle analysis, and predictive maintenance planning.

5.2 Technology Stack

Layer	Technology	Purpose
Frontend	React.js + TypeScript	Component-based UI development with type safety
Styling	Tailwind CSS + ShadCN UI	Responsive utility-first styling and UI components
Routing	React Router	Client-side navigation and protected route management
Backend	Supabase	Cloud backend with PostgreSQL, REST APIs, authentication
Database	PostgreSQL	Relational database for all infrastructure data
Authentication	Supabase Auth	JWT-based session management and role-based access
Storage	Supabase Storage	Cloud storage for issue images and QR code files
Deployment	Vercel	Frontend hosting with automated CI/CD deployment
QR Generation	QR Code Library	Dynamic QR code generation linked to system identifiers
Mobile App	Flutter / React Native	Cross-platform mobile application for fault reporting

Table 5.1: Technology Stack

5.3 Project Modules

Module	Purpose
Mobile App	Scan QR codes and report equipment issues from field locations
Admin Panel	Manage laboratories, systems, complaints, and technician assignments
Authentication	Login and access control for administrators and technicians
QR Management	Generate, manage, and distribute system-specific QR codes
Issue Management	Track all complaints through their complete lifecycle
Notification System	Alert administrators and technicians of new and updated issues

Module	Purpose
Maintenance Tracking	Store and retrieve complete equipment repair and service history
Analytics Dashboard	Generate insights from complaint data for planning and reporting

Table 5.2: Project Modules and Purpose

5.4 Recommended Development Order

The recommended sequence for implementing the system modules is as follows:

11. Database Design: Define all tables, relationships, constraints, and indexes in PostgreSQL.
12. Backend APIs: Implement all REST API endpoints for CRUD operations on each entity.
13. Admin Panel: Build the web-based administrative dashboard with all management modules.
14. QR Generation: Implement QR code generation and linking to system identifiers.
15. Mobile App Scanner: Develop the mobile application with QR scanning capability.
16. Issue Reporting: Implement the complete fault reporting form and submission workflow.
17. Notifications: Integrate automated notification dispatch for all lifecycle events.
18. Analytics Dashboard: Build real-time analytics and reporting capabilities.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project has designed and described a comprehensive Smart Laboratory Equipment Fault Management System that addresses the significant operational challenges faced by educational institutions in managing laboratory equipment maintenance. The system replaces informal, fragmented, and inefficient manual reporting processes with a structured digital workflow accessible through both web and mobile interfaces.

A systematic review of six related works confirmed that while each existing system contributes useful ideas and features, none of them provides the complete, integrated feature set that laboratory fault management in an educational institution requires. The proposed system is the first to simultaneously satisfy all nine evaluation criteria identified in the comparative analysis: fault reporting, real-time tracking, role-based access control, automated notifications, administrative dashboard, data analytics, mobile application, web application, and automated technician assignment.

The system's combination of QR-code equipment identification, automated priority tagging and technician assignment, cluster fault detection, repeated fault alerts, and predictive common fix suggestions makes it a significantly more capable solution than any reviewed work. The technology choices of React.js, TypeScript, Supabase, PostgreSQL, and Vercel provide a robust, scalable foundation that supports both current operational requirements and future enhancement.

The implementation results demonstrate that QR-based infrastructure complaint management can significantly improve operational efficiency within educational institutions. The integration of cloud-based technologies, centralized analytics, and structured complaint workflows creates a scalable digital infrastructure management ecosystem that reduces maintenance delays, improves accountability, simplifies complaint registration, and enhances infrastructure visibility for administrators and technical staff.

The modular architecture and scalable database design also make the system suitable for adaptation in other institutional environments such as universities, corporate offices, hospitals, libraries, and training centers, establishing a practical foundation for modern infrastructure maintenance and operational management systems.

6.2 Future Work

Several directions for future enhancement and extension of the system have been identified:

1. Native Mobile Application:

Developing a dedicated native mobile application using Flutter or React Native will provide students and faculty with a more optimized reporting experience, including offline capability for locations with poor network connectivity. Fault submissions would be queued locally and uploaded when the connection is restored.

2. AI-Based Predictive Maintenance:

The next development phase will extend the predictive suggestion engine with a more sophisticated machine learning model trained on accumulated fault history data. This model will predict equipment failure probability based on usage patterns, maintenance history, and fault frequency, enabling proactive maintenance scheduling before failures occur.

3. Audio Fingerprinting for Equipment Identification:

Future versions may incorporate audio fingerprinting technology to enable automatic identification of equipment by analyzing acoustic signatures. This would provide an alternative identification method for equipment where QR codes are damaged, inaccessible, or impractical.

4. Integration with Institutional Asset Management:

Integration with institution-level asset management systems will allow equipment records to be synchronized automatically rather than maintained manually. This will eliminate data duplication and ensure that fault management records are always aligned with the official equipment inventory.

5. Green Maintenance Scoring:

A Green Maintenance scoring feature is planned that will track the proportion of faults resolved through repair versus replacement, supporting institutional sustainability goals and environmental reporting requirements.

6. Advanced Analytics and Reporting:

Enhanced analytics capabilities including trend analysis, predictive maintenance dashboards, cost tracking, and equipment lifecycle management will be developed to provide administrators with comprehensive decision support tools for infrastructure planning.

7. Hardware-Level Monitoring Integration:

Future integration with IoT sensors and hardware monitoring agents will enable the system to automatically detect faults in real time without requiring manual reporting, creating a truly proactive maintenance ecosystem.

REFERENCES

- [1] Zetian Guo, Yingyue Chen, "Design and Implementation of Laboratory Fault Reporting System Based on Asp.net," International Journal of Information Technology and Computer Science, 2018.
- [2] S. Giridharan, A. Nivedha, V. Vinothini, V. Maha, M. Nivetha Kumari, "Web-Based Computer Maintenance Management System," International Journal of Engineering Research and Technology, 2020.
- [3] Shakir A. Madiyev, "Computer System Maintenance in a Corporate Environment," Journal of Systems and Software Management, 2019.
- [4] Abiraj P, Jaiprasanth S.K, Gunalan K, "Computer System Complaint Tracking System - Web Application," International Research Journal of Engineering and Technology (IRJET), 2021.
- [5] Ramachandra Thirupathi, Ranganatha Rao, T.R. Srinivas, "Design, Development and Implementation of Computerized Maintenance Management Information System (CMMIS)," International Journal of Computer Applications, 2017.
- [6] R. Razali, K.N. Abd Halim and K. Jusoff, "Computer Management System," International Journal of Business and Management, 2009.
- [7] React.js Documentation, Meta Open Source, <https://reactjs.org>, 2024.
- [8] Supabase Documentation, Supabase Inc., <https://supabase.com/docs>, 2024.
- [9] PostgreSQL Documentation, PostgreSQL Global Development Group, <https://www.postgresql.org/docs>, 2024.
- [10] Vercel Deployment Documentation, Vercel Inc., <https://vercel.com/docs>, 2024.

[11] TypeScript Handbook, Microsoft Corporation, <https://www.typescriptlang.org/docs>, 2024.

[12] Tailwind CSS Documentation, Tailwind Labs, <https://tailwindcss.com/docs>, 2024.

[13] Node.js Documentation, OpenJS Foundation, <https://nodejs.org/en/docs>, 2024.

[14] React Native Documentation, Meta Open Source, <https://reactnative.dev/docs>, 2024.

[15] Flutter Documentation, Google LLC, <https://flutter.dev/docs>, 2024.